《菜鸟驿站配送调度系统》项目报告

一、数据结构设计

1. 图（Graph）

实现方式：邻接表

选择理由：

- 校园地图通常为稀疏图，边的数量与节点数在同一数量级。邻接表的空间复杂度为 $O(V+E)$，远优于邻接矩阵的 $O(V^2)$。
- 邻接表便于遍历节点的所有邻边，这是 Dijkstra 算法的核心操作。

核心接口与复杂度：

操作	复杂度	说明
addEdge(u,v,w)	O(1)	在 u 和 v 的链表中各添加一条边
dijkstra(source)	O((V+E) log V)	使用手写最小堆，每个节点入堆一次、出堆一次，每条边松弛一次

操作	复杂度	说明
getDistance(u,v)	O(1)	直接读取 dijktra 计算好的 dist 数组
getPath(u,v)	O(V)	回溯 prev 数组并反转

关键实现细节:

- 边结构体 Edge 包含 u, v, weight，存储于邻接表 graph 中。
- dijkstra 使用“懒删除”策略：当从堆中弹出的节点距离大于已记录距离时跳过，避免手写堆不支持 decrease_key 的限制。
- 节点坐标存储于 nodes 数组，供可视化使用。

2. 最小堆（MinHeap）

实现方式：数组实现的二叉堆，模板化设计（仅用于 HeapNode）。

时间复杂度:

操作	复杂度
push	O(log n)
pop	O(log n)
top	O(1)
empty	O(1)

关键实现：

- siftUp：新元素插入末尾后向上调整，每次与父节点比较，若小于父节点则交换。
- siftDown：删除堆顶后，将末尾元素移至堆顶并向下调整，每次与较小的子节点交换。
- 完全满足 Dijkstra 算法的需求，且符合“手写实现、不使用 STL”的要求。

3. 循环队列（CircularQueue）

实现方式：基于数组的循环队列，支持自动扩容。

时间复杂度：

操作	复杂度
enqueue	O(1) 均摊（扩容时 O(n)）
dequeue	O(1)
front	O(1)

空间复杂度：O(n)，n 为当前容量。

设计要点：

- 用两个指针 front 和 rear 维护环形缓冲区。
- 当队列满时，容量加倍，将元素复制到新数组并重置指针。
- 存储类型为 Parcel，用于驿站待派送包裹的管理（题目要求使用队列维护配送任务）。

二、各任务设计思路与正确性说明

T1 最短路查询

问题：查询地图上任意两点之间的最短距离和路径。

算法：Dijkstra 单源最短路径（手写最小堆优化）。

思路：

1. 从源点 u 出发，初始化距离为 0，其余节点为 ∞。
2. 使用手写最小堆维护“当前已发现但未处理的最近节点集合”。
3. 每次取出堆顶节点，若其距离已过时（> 记录的 dist），跳过；否则对其每条邻边执行松弛操作，更新邻居的距离和前驱节点。
4. 算法结束后，dist[v] 即为最短距离，prev 数组可回溯路径。

正确性：Dijkstra 算法的标准证明——贪心选择当前最短的未处理节点，因为边权非负，该节点的距离不可能再被更新。

用途：T1 本身可查询任意两点最短距离，也是 T2/T3 计算路径和时间的基础。

T2 最小化不满意度之和

问题：在容量约束和包裹到站时间限制下，最小化所有包裹的“不满意度之和”——即 Σ(送达时间 - 到站时间)。

算法框架：贪心分批 + 批次内路径优化。

思路：

1. **排序**：按包裹到站时间 S 升序排列，保证先到的包裹优先被考虑。
2. **贪心分批**：依次将包裹加入当前批次，直到累计重量超过小车容量 w_lim，则保存当前批次并新建下一批。这保证每个批次都不超重，且批次之间出发时间自然递增。
3. **批次内路径规划**：
    - 若该批次的目的地数量 $\leqslant 10$，使用**全排列**枚举所有可能的访问顺序，选取使该批次不满意度之和最小的顺序。
    - 若 > 10，使用**最近邻贪心**：从驿站出发，每次选择距离当前节点最近的未访问目的地。
4. **出发时间约束**：每趟的出发时间必须不早于该批所有包裹的到站时间 S 的最大值，也不早于小车完成上一趟返回的时间。
5. **不满意度累加**：对每个包裹，计算送达时间 - S，累加得到总不满意度。

正确性：

- 分批策略遵循“尽量装满”的原则，减少趟数从而

减少总行驶时间，有助于降低等待时间。

- 批次内全排列保证在小规模下找到最优访问顺序；对于大规模，最近邻是一种经典且高效的近似算法，虽然不一定最优，但能给出合理的解。
- 出发时间约束严格遵守题目要求，确保包裹不会在到站前被配送。

优缺点：

- 优势：实现清晰，分批逻辑简单，小规模时精确优化。
- 局限：最近邻可能陷入局部最优，但不满意度总和仍然可控。

T3 带容量的运送成本

问题：在包裹随时可送（S=0）的条件下，考虑小车自重和车上包裹重量对成本的影响，使总成本最小化，并统计超时包裹数。

成本模型：每段路的成本 = 路段长度 × (小车自重 + 该段行驶时车上仍未卸下的包裹总重)。

算法框架：贪心分批 + 加权贪心路径规划。

思路：

1. **排序**：按包裹的 deadline T 升序排列，让紧急的包裹优先被安排。
2. **贪心分批**：与 T2 相同的容量约束分批。
3. **批次内路径规划**：使用“重量/距离”评分贪心——从当前节点出发，每次选择 重量 / 距离 最大的目的地作为下一站。这样可以优先去那些“重且近”的点，让重包裹尽早卸下，降低后续路段的载重成本。
4. **成本计算**：逐段累加：segDist * (car_weight + curLoad)，其中 curLoad 在每卸一次货后减少。
5. **超时统计**：每个包裹的送达时间与 deadline 比较，超过则计数 +1。

正确性：

- 按 deadline 排序有助于减少超时；贪心路径规划优先卸重货符合成本模型的直观：越重的包裹越早卸下，后续行驶成本越低。
- 该策略不是全局最优，但在 $O(k^2)$ 时间内给出了合理方案，适合节点较多的情况。

T4 退货回收

问题：给定退货地点，规划最短回路取回所有货物。

算法：

- 节点数 ⩽ 20：状态压缩 DP 求精确最优解，复杂度 $O(n^2 \cdot 2^n)$。
- 节点数 > 20：最近邻构造初始解 + 2-opt 局部搜索改进，复杂度 $O(n^2 \cdot \text{iteration})$。

正确性：状态压缩 DP 保证精确；2-opt 通过交换两条边来消除交叉，能在多项式时间内显著改进路径质量。

T5 双车协同

思路：

1. **分配**：按包裹目的地到驿站的距离降序排列，依次将包裹分配给当前累计重量较小的车辆，实现负载均衡。
2. **单车调度**：每辆车复用 T3 的 planT3ForSingleCar 算法，独立规划行程。
3. **汇总**：两车成本相加，超时数相加。

正确性：负载均衡有助于两车任务量相近，避免一车过忙而另一车闲置；单车调度基于 T3 的贪心策略。

T6 更优策略对比

实现：实现了三种 T3 路径策略的对比框架：

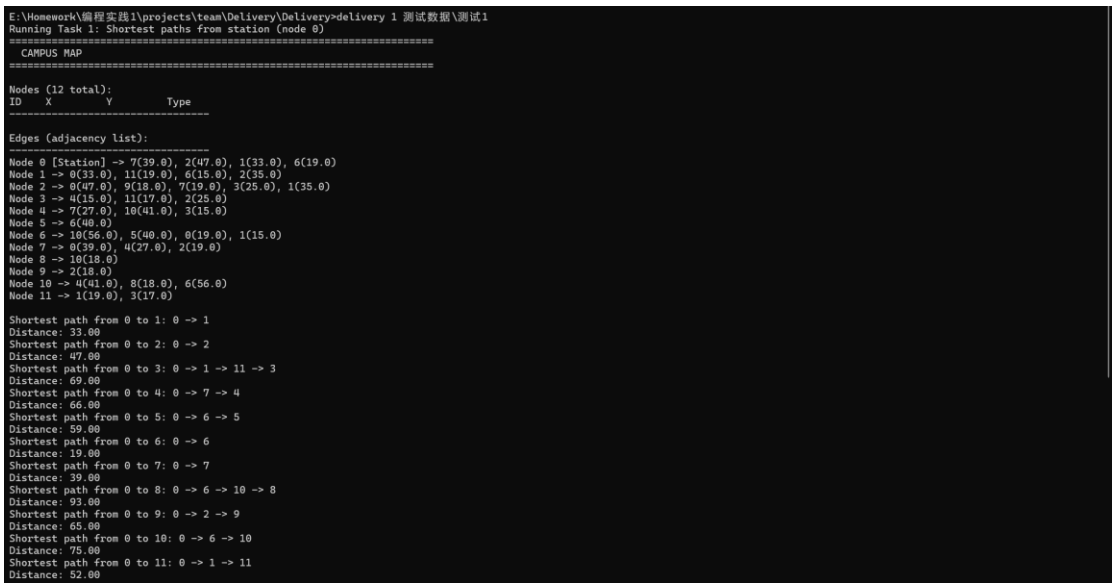
- 基线策略 1：最近优先（Nearest First）
- 基线策略 2：最早截止时间优先（EDD）
- 改进策略：综合评分（重量/距离 × 1/deadline）

T2 对比了最近优先与重量/距离评分策略。

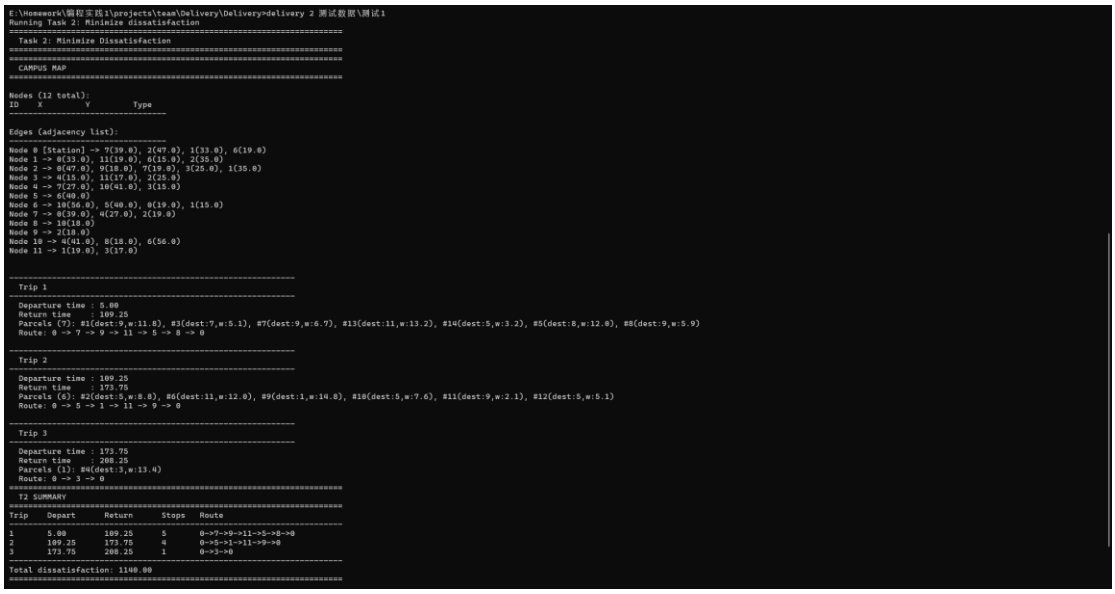
结论：在 deadline 差异较大的数据上，改进策略优于纯最近优先，因为它兼顾了紧急程度；但在 deadline 均很宽松的均匀数据上，几种策略差异不大。

三、测试结果

3.1 T1 最短路查询（以测试数据 1 示例）



3.2 T2 不满意度最小化



3.3 T3 运送成本



3.4 T4 退货回收

```
E:\Homework\CodingPractice1\projects\team\Delivery>delivery 4 Delivery\测试数据\测试3
Running Task 4: Return pickup (TSP)
==================== Task 4: Return Pickup ====================
Pickup order (node IDs): 18 -> 3 -> 14 -> 11 -> 6
Total travel distance: 66.00
================================================================
```

3.5 T5 双车协同

```
E:\Homework\编程实践1\projects\team\Delivery\Delivery>delivery 5 测试数据\测试1
Running Task 5: Dual-vehicle coordination
==================== Task 5: Dual-Vehicle Coordination ====================
Car 1 gets 7 parcels, total weight: 55.00
Car 2 gets 7 parcels, total weight: 66.70

--- Car 1 Trips ---
  Trip 1: 3 2 11 1 6 14 5 | Cost: 18924.70 | Overtime: 1

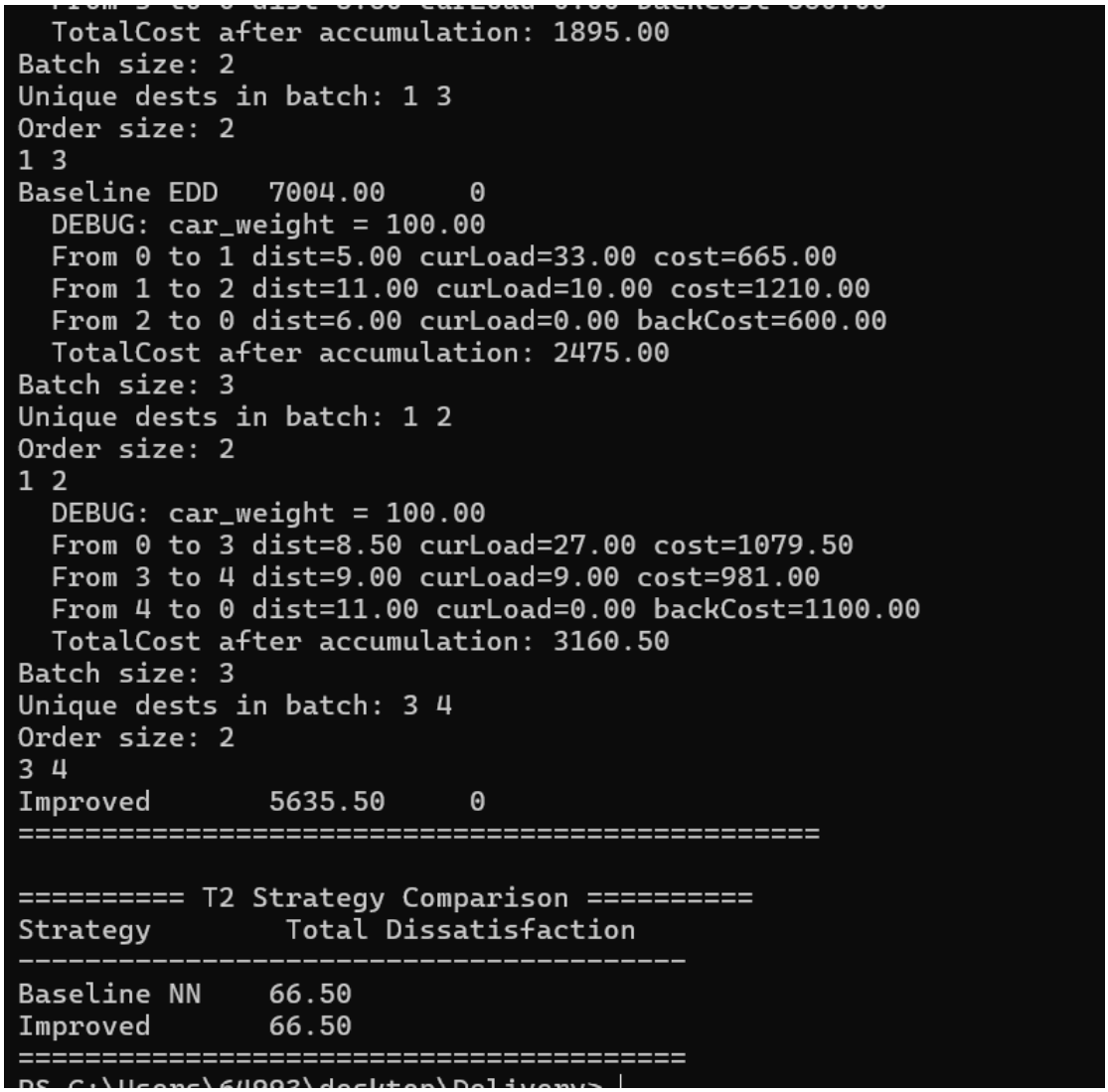
--- Car 2 Trips ---
  Trip 1: 12 10 13 8 9 4 | Cost: 13038.60 | Overtime: 0
  Trip 2: 7 | Cost: 3295.50 | Overtime: 0

Total cost (both cars): 35258.80
Total overtime parcels: 1
============================================================================
```

3.6 T6 更优策略

```
PS C:\Users\64993\desktop\Delivery> ./Delivery 6 Delivery/测试数据/T6
Running Task 6: Strategy comparison on given data

========== T3 Strategy Comparison ==========
Strategy        Total Cost      Overtime Parcels
-----------------------------------------------
  DEBUG: car_weight = 100
  From 0 to 1 dist=5 curLoad=33 cost=665
  From 1 to 2 dist=11 curLoad=10 cost=1210
  From 2 to 0 dist=6 curLoad=0 backCost=600
  TotalCost after accumulation: 2475
Batch size: 3
Unique dests in batch: 1 2
Order size: 2
1 2
  DEBUG: car_weight = 100
  From 0 to 3 dist=8.5 curLoad=27 cost=1079.5
  From 3 to 4 dist=9 curLoad=9 cost=981
  From 4 to 0 dist=11 curLoad=0 backCost=1100
  TotalCost after accumulation: 3160.5
Batch size: 3
Unique dests in batch: 3 4
Order size: 2
3 4
Baseline NN    5635.50     0
  DEBUG: car_weight = 100.00
  From 0 to 1 dist=5.00 curLoad=39.00 cost=695.00
  From 1 to 2 dist=11.00 curLoad=27.00 cost=1397.00
  From 2 to 3 dist=8.00 curLoad=17.00 cost=936.00
  From 3 to 4 dist=9.00 curLoad=9.00 cost=981.00
  From 4 to 0 dist=11.00 curLoad=0.00 backCost=1100.00
  TotalCost after accumulation: 5109.00
Batch size: 4
Unique dests in batch: 1 2 3 4
Order size: 4
1 2 3 4
  DEBUG: car_weight = 100.00
  From 0 to 1 dist=5.00 curLoad=21.00 cost=605.00
  From 1 to 3 dist=4.00 curLoad=10.00 cost=440.00
  From 3 to 0 dist=8.50 curLoad=0.00 backCost=850.00
  TotalCost after accumulation: 1895.00
```



四、总结

本项目实现了一个完整的校园快递配送调度系统，核心算法基于 Dijkstra 最短路和贪心调度策略，能够处理容量约束、时间窗约束，并计算不满意度和能耗成本。

项目严格遵循了“手写数据结构”的要求（最小堆、循环队列），并在选做部分探索了 TSP、双车协同和多

策略对比，完成了所有必修和部分选修任务